

# EEG Foundation Models for Generating High-Quality Embeddings

## BrainBERT

**Architecture:** BrainBERT is a Transformer-based encoder (BERT-style) with a shallow decoder (two linear layers) for reconstruction <sup>1</sup>. It operates on *single-channel time-frequency* inputs: each EEG channel's signal is transformed into a spectrogram (e.g. STFT or Superlet) and divided into patches, which the Transformer encodes <sup>2</sup>. The shallow decoder then predicts the masked spectrogram patches from the encoded representation.

**Pretraining:** BrainBERT was pretrained with a masked reconstruction objective on ~4.5k channel-hours of intracranial EEG (iEEG) data (the *Brain TreeBank* dataset) <sup>3</sup>. It masks ~10% of spectrogram patches (5% in time, 5% in frequency) and learns to reconstruct them, akin to BERT's masked language modeling but for neural spectral patches <sup>4</sup> <sup>5</sup>.

**Strengths:** Despite its modest scale (43M parameters), BrainBERT demonstrated strong transfer learning: linear probes on its embeddings rivaled fully trained deep networks on neural decoding tasks, and fine-tuning required as little as 15% of the data to reach full model performance <sup>6</sup>. It generalized to new subjects and electrode locations (though within the same data distribution) <sup>7</sup>. As an early EEG foundation model, it showed that self-supervised pretraining on neural data can significantly boost decoding accuracy with limited labels <sup>8</sup>. BrainBERT's focus on spectrogram patches also incorporates frequency-domain information inherently.

**Weaknesses:** BrainBERT was trained on a single iEEG dataset with limited diversity (only intracranial signals from 9 subjects) <sup>9</sup>. This narrow domain means its learned features may be less directly applicable to scalp EEG or very different tasks. Its architecture ignores cross-channel relationships (each channel's spectrogram is processed independently), so it doesn't explicitly model spatial (electrode-to-electrode) dependencies. Moreover, its output is channel-specific: each input channel yields a sequence of patch embeddings, so additional pooling or fusion would be needed to produce a unified multichannel representation. In scale, BrainBERT is relatively small; while that aids fine-tuning on small data, it may limit the complexity of patterns it can represent compared to newer, larger models.

**Output:** BrainBERT produces a sequence of embeddings per channel spectrogram (one embedding per spectrogram patch). For example, a 5-second window per channel might be divided into patch tokens (time-frequency tiles), each yielding a  $d$ -dimensional embedding (with  $d$  likely on the order of a few hundred). To use BrainBERT for multimodal integration, one could concatenate or average the embeddings across channels (or use a learned pooling) to get a single vector per time-step. Alternatively, a special "[CLS]" token embedding could represent the whole window if the model is configured that way (the original paper used linear probes on the patch embeddings, rather than a single global token) <sup>10</sup>.

**Code/Checkpoints:** BrainBERT’s code and pretrained weights are available (MIT CSAIL) <sup>11</sup>. The authors released the model on GitHub <sup>11</sup>, making it reasonably accessible for extension. The implementation is in PyTorch, and being an ICLR 2023 model, it’s well-documented. However, as it’s focused on iEEG, adapting it to scalp EEG might require additional fine-tuning or spectral preprocessing adjustments.

## CBraMod (Criss-Cross Brain Foundation Model)

**Architecture:** CBraMod introduces a *criss-cross Transformer* backbone that separates spatial and temporal attention streams <sup>12</sup>. EEG data are divided into *patches* defined by segments of time *per channel* (e.g. 1-second segments from each channel) <sup>13</sup> <sup>14</sup>. In each Transformer block, CBraMod performs **parallel** multi-head self-attention: half the heads attend *across channels within the same time interval* (spatial attention) and the other half attend *across time within the same channel* (temporal attention) <sup>13</sup>. By alternating and combining these, the model captures heterogeneous dependencies more effectively than “full” attention over all patches <sup>15</sup> <sup>12</sup>. CBraMod also uses an **Asymmetric Conditional Positional Encoding (ACPE)** scheme: a lightweight convolutional positional encoder that encodes patch positions in a way that can adapt to varying channel layouts <sup>12</sup> <sup>16</sup>. In practice, this means CBraMod can handle different EEG montages (number and arrangement of channels) without losing positional context, unlike fixed electrode ID embeddings. The backbone has 12 Transformer layers, with 200-dimensional embeddings (hidden size  $d=200$ ), 8 heads (4 spatial + 4 temporal), and an initial patch embedding module (1D convolutions) to encode each 1-second segment per channel into the  $d$ -dim vector <sup>17</sup>. A simple linear “reconstruction head” on top of the encoder is used during pretraining to map embeddings back to patch signals <sup>18</sup>.

**Pretraining:** CBraMod is pretrained as a **masked autoencoder**, with *patch-based masked EEG reconstruction* as the objective <sup>16</sup>. The model was trained on the Temple University Hospital EEG (TUEG) corpus – one of the largest public EEG datasets – containing ~27,000 hours of clinical scalp EEG from ~15k subjects (after preprocessing, >9,000 hours of clean data were used for training) <sup>19</sup> <sup>20</sup>. During pretraining, a random subset of the EEG patches (e.g. 40% as in typical MAE setups) is masked, and CBraMod must reconstruct those missing patches from the unmasked ones using an MSE loss <sup>18</sup> <sup>21</sup>. This teaches the model to build holistic EEG representations. Notably, the EEG was segmented into 30-second samples (each sample yields  $30 \times C$  patches, with  $C=19$  channels after standardization) and patch length was 1 s, so the context per forward pass was 30 seconds of multichannel EEG <sup>17</sup>.

**Strengths:** CBraMod achieves **state-of-the-art** performance across a wide range of EEG tasks and datasets <sup>22</sup>. In evaluations on 10 diverse BCI tasks (12 public datasets, including emotion classification, motor imagery, seizure detection, sleep staging, etc.), it outperformed prior models, demonstrating strong generalization <sup>22</sup>. Its criss-cross attention architecture explicitly respects EEG’s structure: by separating spatial and temporal processing, it can capture within-channel temporal dynamics and between-channel interactions more effectively than a standard Transformer that mixes both indiscriminately <sup>15</sup> <sup>12</sup>. This leads to better robustness when channel configurations or signal lengths differ, addressing a key pain point in applying one model to many EEG datasets <sup>23</sup> <sup>24</sup>. Another strength is **format adaptability**: thanks to ACPE, CBraMod can be fine-tuned on new EEG setups (different electrode placements or counts) with minimal architectural change, a significant advantage for multimodal integration where EEG channel setups might vary by dataset. The model’s size is moderate (~12 layers, 200-d hidden — on the order of tens of millions of params, not counting the small conv layers), making it feasible to fine-tune or to integrate as a sub-module in a larger multimodal model. Finally, CBraMod’s code is openly available in PyTorch <sup>22</sup> and the paper provides thorough documentation, so using it as a foundation is developer-friendly.

**Weaknesses:** Being a newly introduced model (ICLR 2025), CBraMod’s *pretrained weights* may not yet be as battle-tested as some older models. Its training data, while large, is solely scalp EEG (clinical data from TUEG) <sup>19</sup> — it does not include intracranial signals or non-EEG modalities, so its representations are specific to scalp electrophysiology. If the target application involves very different EEG characteristics (e.g. intracranial signals or MEG), CBraMod might require further pretraining or adaptation. The design of one patch = 1 s of one channel means the model processes a large number of tokens for long recordings (e.g. 19 channels \* 30 patches = 570 tokens for 30 s), which is computationally heavy; in a real-time multimodal system with longer sequences, this may pose memory challenges (though techniques like downsampling or patch merging could mitigate this). Also, while separate spatial/temporal attention is great for interpretability (one can inspect, for instance, which channels attend to which at a given time), it imposes a structural bias — in scenarios where some cross-dependency spans both space and time jointly, the parallel attention might need multiple layers to capture it (the model alternates spatial/temporal mixing per layer). In summary, CBraMod trades some architectural simplicity for inductive bias; this is usually a net positive for EEG, but it’s a design less conventional than standard Transformers, which might complicate certain modifications.

**Output:** CBraMod’s encoder produces a set of *patch representations*  $r_{\{(ch,t)\}}$  for each patch (each channel–time segment) <sup>25</sup>. So the raw output shape is (Batch, Patches, 200), where Patches = (number of channels × number of time-segments per sample). For example, a 30 s, 19-channel EEG sample yields 570 patch embeddings of size 200 each. For feeding into a multimodal model expecting  $(B,T,D)$ , we need one embedding per time-step. A straightforward approach is to **aggregate across channels** for each time segment. Since CBraMod already separates time segments (temporal attention handles correlations along each channel’s 30 patches), we can take all patch embeddings at time  $t$  (across the  $C$  channels) and combine them into one  $D$ -dimensional vector. This could be done by averaging the channel embeddings at each time, or by an additional small attention/pooling layer that produces a “time- $t$  summary” vector. Notably, CBraMod’s spatial attention heads already mix information among patches of the same time, so each channel’s patch embedding  $r_{\{(ch,t)\}}$  contains some information from other channels at time  $t$ . Thus, even a simple mean-pooling across channels at each time index can yield a rich representation for that time window. This would produce an output of shape  $(B, T_{\text{seg}}, D)$  — e.g.  $(B, 30, 200)$  if  $T_{\text{seg}}=30$  one-second segments — which could then be aligned with video frames or audio features (possibly by interpolating or buffering if modalities have different rates). If finer temporal resolution is needed (e.g. one EEG embedding per video frame of 33 ms), one could train a smaller CBraMod patch size or use the existing model and interpolate the output in time (though 1 s is already quite fine for EEG in many contexts).

**Code/Checkpoints:** CBraMod’s PyTorch implementation and pre-trained weights are publicly released <sup>22</sup>. The repository provides training and fine-tuning scripts, and the model is relatively easy to load and use. Given its recency, the documentation is up-to-date, and the authors explicitly designed it for use “out of the box” on new tasks (aligning with the foundation model ethos). This makes CBraMod a strong candidate for building an EEG module in a larger multimodal pipeline.

## Brant (Foundation Model for Intracranial Neural Signals)

**Architecture:** Brant (NeurIPS 2023) is a large-scale foundation model designed originally for **intracranial EEG (SEEG)** signals <sup>9</sup>. It employs a dual-encoder architecture: a **temporal encoder** to capture long-range time dependencies in each channel, and a **spatial encoder** to capture correlations among channels <sup>26</sup>. In practice, Brant first processes raw multi-channel time series with the temporal encoder (likely a Transformer

or similar) that operates within each channel's sequence, producing intermediate features that encode the time dynamics. These features are then fed into a spatial encoder (another Transformer encoder) that treats the channels as a sequence, learning inter-channel relationships <sup>26</sup>. By separating these stages, Brant ensures that the model learns both within-channel patterns (like rhythms, transients over time) and cross-channel patterns (like phase synchrony or multichannel propagation events). Brant also incorporates **time-frequency information**: the design mentions combining information in both domains <sup>27</sup>, which suggests that the input embedding may include features such as channel spectrograms or frequency-domain features alongside time-domain signals. Possibly, Brant's temporal encoder or input layer is enriched by spectral features (the specifics in the paper indicate a spectral representation complement, but not a full spectrogram per channel) <sup>28</sup>. A simple linear decoder head on top of the spatial encoder outputs the reconstruction or prediction, depending on the pretext task <sup>29</sup>. The model was released in multiple sizes: the largest ~500M parameters, and smaller variants of 68M, 104M, and 249M using the same architecture on the same data <sup>30</sup>.

**Pretraining:** Brant was pretrained on 281k channel-hours of iEEG from a single, sizable SEEG dataset (9 subjects with long recordings) <sup>9</sup>. Its self-supervised objective was **masked signal reconstruction** (a patch-based autoencoding similar to BrainBERT's, but on raw time series) <sup>31</sup> <sup>29</sup>. In Brant's case, segments of the multichannel time series were masked, and the model learned to predict the missing portions. Thanks to the spatial encoder, it could even mask entire channels or large time gaps and still reconstruct them using information from other channels and time steps. Downstream, Brant was evaluated on a variety of tasks relevant to intracranial data: short-/long-term **signal forecasting**, frequency-phase forecasting (predicting future oscillation phases), **imputation** of missing data, and **seizure detection** <sup>32</sup>. The model showed consistently strong performance across these tasks, outperforming specialized models in each case (since it learned a general representation of neural dynamics) <sup>33</sup>.

**Strengths:** Brant was a breakthrough as one of the first very-large param neural signal FMs. It explicitly introduced a *spatial encoder module* – a novel component at the time – to handle channel relationships <sup>26</sup>, which was shown to improve performance over earlier models that lacked dedicated spatial modeling. Its ability to accept **variable channel configurations** is important: Brant can be fine-tuned on data with different electrode setups because the spatial encoder isn't tied to a specific number of channels (it processes whatever channel sequence is provided, up to positional embeddings) <sup>9</sup>. This flexibility is useful in multimodal settings, as you could drop or add channels without redesigning the model. Brant's inclusion of time-frequency features means it can leverage spectral characteristics of EEG (important for things like seizure rhythms or sleep oscillations) alongside time-domain trends <sup>28</sup>. In terms of results, Brant achieved state-of-the-art or highly competitive accuracy on diverse tasks including forecasting and detection, demonstrating it learned a robust latent representation of brain signals <sup>33</sup>. It essentially validated the concept that a single foundation model can support many neural signal tasks. The availability of pretrained weights (including a 500M-param model) <sup>34</sup> and smaller variants means one can choose a scale appropriate for the computational budget. Documentation from the authors (ZJU Brainnet lab) is solid; they provide a GitHub and even a HuggingFace repository <sup>35</sup> <sup>36</sup>.

**Weaknesses:** A major limitation of Brant is that its pretraining data, though large in hours, came from only 9 subjects and all from the same data source <sup>9</sup>. This *lack of diversity* can lead to overfitting to that dataset's characteristics. Indeed, the model might have learned idiosyncrasies of that specific cohort or recording setup. Thus, Brant's generalization to other EEG/SEEG datasets or to scalp EEG isn't guaranteed – it may require further pretraining or careful fine-tuning on new data. The **scale** of the full model (500M) is also a double-edged sword: it has high capacity but is computationally expensive to train and use. Fine-

tuning such a large model demands considerable GPU memory and time, which might be overkill if one's EEG dataset is smaller or if one only needs an embedding module in a larger system. In contrast to CBraMod, Brant did not explicitly have a solution for varying reference schemes or electrode positions beyond providing positional encodings for channels; it assumes some consistent ordering or at least that a model pre-trained on one montage will transfer to another, which might not hold for scalp EEG with very different channel layouts. Another weakness is that Brant's architecture (temporal then spatial encoder) processes channels sequentially in the second stage, which could be inefficient if channel count is high (the spatial attention is quadratic in channel count). If one had, say, 128-channel EEG, the spatial encoder might be heavy. Also, Brant's output during pretraining is at the *raw signal level* (it reconstructs signals), so its learned embedding isn't explicitly distilled – one might need to decide at which layer to tap the representation for downstream use. Typically, one would use the output of the spatial encoder (before the linear decoder) as the multi-channel embedding. This output has shape (batch, channels, features) or (batch, time, features) depending on how the reconstruction is formulated. The original Brant likely outputs a full time-series per channel (for reconstruction); to get a (T,D) format, an adaptation would be needed (see below).

**Output:** In Brant's architecture, after the temporal and spatial encoders, you effectively have a representation for each channel and each time step (or each patch of time) before decoding. For instance, if Brant processes a window of  $T$  time steps per channel as input (possibly segmented into patches internally), the temporal encoder yields something like  $(\text{channels}, T, d)$  features, and the spatial encoder then mixes channels, resulting in an output of similar shape  $(T, \text{channels}, d)$  or  $(\text{channels}, T, d)$  (order aside). The linear decoder would then map each of those features back to a signal value for reconstruction. To use Brant for embeddings, we can take the spatial encoder's output and reinterpret it. One approach is to swap the order (treat time as the primary sequence): you would end up with an embedding per time slice (with information integrated from all channels at that time). In effect, Brant can produce an embedding sequence over time, where each time-index (or time patch) now has a vector that encodes multi-channel activity at that moment. This is suitable for a cross-modal scenario: you could set  $T$  to correspond to your desired sequence length (e.g. one embedding per 0.5 s or 1 s). If Brant was pretrained with a certain window length (say 10 s windows), you might use that same window length and take the embeddings at each smaller patch within to feed the multimodal model. If finer resolution is needed than what Brant was trained on, one might fine-tune the model to shorter context. In summary, Brant can yield  $(B, T, d)$  outputs by using the spatial encoder's output per time token. Minor adaptation (like adding a projection) might be needed if we want exactly one vector per *whole window* (Brant doesn't inherently produce a single "CLS" token for the window). For multimodal integration, one likely would use a sliding window or chunking approach: e.g. run Brant on 10-second segments, get an embedding for each 1-second patch within, aligning those with 1-second video or audio features.

**Code/Checkpoints:** The authors provided code (PyTorch) and even pre-trained weights for Brant <sup>36</sup>. The model is accessible on HuggingFace and their project page, though the largest weight (500M) may require accepting a model license. The code includes the tokenization and the dual-encoder implementation, which can be leveraged for custom data. However, given Brant's intracranial focus, using its checkpoint for scalp EEG might require further fine-tuning on scalp data to adjust, but using it as an initialization for intracranial EEG tasks or similar domains is very promising.

## Brant-2 (Next-Generation Brain Signal Foundation Model)

**Architecture:** Brant-2 (2024) is an *evolution of Brant* designed to be **bigger, more general, and multimodal in EEG/SEEG terms**. It retains a similar overall architecture (patch-based Transformer with temporal and spatial encoder stages) but introduces some new features for scalability and robustness <sup>37</sup> <sup>38</sup>. The input is first divided into **subseries-level patches** (small consecutive time chunks, e.g. a patch might be 2 s of data) <sup>39</sup> <sup>40</sup>. These patches go through a **data augmentation module** that randomly resamples the time (to augment different sampling rates) and randomly subsets/shuffles channels (to augment different montages), increasing diversity during training <sup>41</sup> <sup>42</sup>. Some patches are masked (for reconstruction), and a portion of the sequence is designated as “future” (for forecasting) <sup>43</sup> <sup>40</sup>. Brant-2 then creates input embeddings by **combining time- and frequency-domain information**: each patch of signal is converted not only into a time-domain feature but also includes frequency features (likely via an FFT or learned spectral encoder), and these are fused (details suggest a multi-branch embedding module) <sup>44</sup> <sup>45</sup>. Once embedded, patches are fed *sequentially through a temporal encoder then a spatial encoder* (much like Brant) <sup>45</sup> <sup>46</sup>. Notably, Brant-2 uses some improved Transformer components: e.g. RMSNorm normalization and a **multi-FFN transformer block** (as hinted in the paper) to enhance learning stability for such a large model <sup>47</sup>. The output of the encoder is used by two heads: one that linearly reconstructs the masked patches (MAE style) and one that predicts the future patch embeddings (i.e. forecasts next patches) <sup>44</sup> <sup>40</sup>. Brant-2 is **massive** – approximately **1 billion parameters** – making it the largest EEG/SEEG foundation model to date <sup>48</sup> <sup>49</sup>. It’s essentially scaled to the level of a small language model, which is unprecedented in this field.

**Pretraining:** Brant-2 was trained on an extremely large and diverse corpus: *nearly 4 TB* of brain signals from **15,000+ subjects**, combining both scalp EEG and intracranial EEG (SEEG) data <sup>48</sup> <sup>50</sup>. This includes multiple datasets and covers a wide range of conditions (sleep, epilepsy, etc.). The training tasks were **dual objectives**: (1) **masked patch prediction** (like Brant’s and CBraMod’s reconstruction, to learn representations of current context), and (2) **future patch forecasting** (predicting upcoming neural activity from past context) <sup>44</sup> <sup>50</sup>. The model uses a context of 16 patches and tries to predict the next 4 patches, for example <sup>51</sup>. These two tasks force Brant-2 to learn both a good encoding of present signals and a sense of temporal dynamics (like a generative model of the signal’s evolution). The loss is a combination of reconstruction MSE and forecasting MSE <sup>52</sup>. By training on both invasive and non-invasive data, Brant-2 aims to be a **universal** brain-signal model. The training process was computationally intensive (100+ hours on high-end GPUs) <sup>53</sup>. After pretraining, Brant-2 was evaluated on an extensive suite of tasks (likely everything from emotion recognition to seizure prediction, as suggested) and demonstrated strong generalization to *both* EEG and SEEG scenarios <sup>54</sup> <sup>44</sup>. It’s touted as the first “off-the-shelf” model covering both domains, maintaining performance even with few labels <sup>55</sup> <sup>56</sup>.

**Strengths:** Brant-2’s primary strength is its **unprecedented scale and scope**. With an order of magnitude more data and parameters than earlier models, it captures a wide array of neural patterns. It is robust to variations in sampling rate, window length, and channel configuration, thanks to the built-in augmentation strategies during pretraining <sup>41</sup> <sup>42</sup>. This means Brant-2 can likely handle different EEG devices or protocols (e.g. 128-channel high-density EEG at 1kHz or 19-channel EEG at 250Hz) better than a model trained on a fixed setup (Brant-2 explicitly was designed to *not* be tied to one sampling rate or montage) <sup>41</sup> <sup>42</sup>. The integration of time and frequency information means the embeddings are rich: the model doesn’t have to infer all frequency content from time samples alone, since it’s explicitly given spectral features too. This could improve performance on tasks like sleep stage or pathology detection, where certain frequency bands are crucial. By training to forecast the future, Brant-2 gains a sort of predictive understanding of

brain signals; this can improve its ability to represent temporal context (useful in a multimodal setting if aligning with future or past context of video). In evaluations, Brant-2 showed excellent generalization across tasks, outperforming the original Brant and other contemporaries, and it remained effective even when fine-tuned with limited labels <sup>57</sup> <sup>50</sup> . Another strength is that **code and pretrained weights are available** <sup>58</sup> , and given the authors' thorough experiments, there's strong documentation on how to fine-tune it for various tasks. In summary, Brant-2 is the most *comprehensive* EEG/SEEG foundation model currently – if one has the computational resources, it provides a powerful starting point.

**Weaknesses:** The obvious weakness is the **resource requirement**. At 1B parameters, Brant-2 demands significant GPU memory for inference and fine-tuning. Incorporating it into a multimodal model with video (which itself might be large, e.g. a Vision Transformer or video model) could be challenging. Some form of model distillation or parameter reduction might be needed to make it practical. Also, while Brant-2's diversity is a plus, it means the model might include a lot of capacity focused on tasks or signal types that might not be relevant to a particular user's application (e.g. if your use-case is only scalp EEG emotion recognition, the SEEG-trained aspects of Brant-2 are extra baggage). Another subtle point: mixing EEG and SEEG could theoretically introduce a risk that the model's representations merge features of both in a way that might not be optimal if you only care about one modality – although the intention is to generalize, one has to be mindful that invasive signals have different characteristics (high-frequency content, less noise) than scalp EEG. The model's training objectives (reconstruction + forecasting) are more complex than a simple autoencoder; this yields richer representations but also might complicate the fine-tuning: one has to decide how to handle the dual-headed nature if reusing the model. In practice, one might drop the forecasting head and just use the encoder weights for feature extraction. Finally, Brant-2, being very new (as of 2024), may not yet have many third-party evaluations; its true generalization will be proven as the community tests it on more datasets. There could be corner cases (e.g., neonatal EEG, or very short events) that we don't know its performance on.

**Output:** Brant-2's encoder produces **contextual patch embeddings**. For a given input sequence (look-back window of, say, 16 patches), the encoder (temporal+spatial) yields an output for each patch in that context <sup>59</sup> <sup>46</sup> . Suppose each patch corresponds to 2 seconds of EEG; then 16 patches = 32 seconds of input, and the model outputs 16 embeddings (plus it forecasts 4 future embeddings, which are separate). Each embedding is a vector (the hidden size isn't explicitly stated, but likely on the order of 768 or 1024 given the model size). So you get an output shape (B, 16, d). These can be viewed as 16 sequential embeddings covering the time span. To integrate into a multimodal pipeline expecting (B,T,D): you could treat those 16 as the "T" dimension (perhaps each as a timestep of ~2s). If your multimodal model expects a certain fixed T (e.g. T video frames), you might slide the Brant-2 window or adjust patch length accordingly. Another idea is to fine-tune Brant-2 in a streaming way: allow it to process continuous data and output a sequence of patch embeddings as it moves along time. For precise alignment with video frames, one might need to interpolate between patch embeddings or even downsample video. If video operates at 30 Hz and Brant-2 at 0.5 Hz per patch (with 2s patches), a strategy is to repeat each EEG embedding for the corresponding span of frames, or use cross-attention that can attend from multiple frames to one EEG embedding spanning those frames. Gated cross-attention (like in e.g. DeepMind's Perceiver/Flamingo) can handle mismatched sequence lengths by attending broadly. In any case, Brant-2 provides *time-indexed embeddings* over long contexts. The **dual objective** also means the model could produce not just current embeddings but could internally generate future ones; however, for simplicity, you'd likely only use the present embeddings for cross-modal alignment. If needed, one could ignore the forecasting part entirely and just use the mask-reconstruction pretrained encoder as a standard masked autoencoder encoder.

**Code/Checkpoints:** Yes, Brant-2's authors have released the code and pretrained model (Apache-2.0 license) <sup>58</sup>. The GitHub `yz673/Brant-2` contains training scripts and model definitions, likely in PyTorch given the lab's other projects. The model weights (1B) are hefty, but they can be downloaded and loaded with HuggingFace's transformers or a provided loader script. Given the model's complexity, the repository documentation is valuable for understanding input formatting (patching, augmentation settings) if you want to use the model as-is. For a production or research multimodal model, one might use Brant-2's weights to initialize an EEG branch and then fine-tune that branch alongside a video/audio branch.

## BIOT (Biosignal Transformer for Cross-Data Learning in the Wild)

**Architecture:** BIOT (NeurIPS 2023) is a lightweight EEG foundation model with an **encoder-only Transformer** architecture and a focus on *efficient tokenization* of signals <sup>60</sup>. It uses a **linear attention** mechanism (e.g. linear Transformers or other efficient attention variants) to reduce computational complexity, which is how it remains so small (3.3M params) while handling long sequences <sup>61</sup>. The key innovation in BIOT is how it tokenizes multi-channel time series: each channel's signal is broken into fixed-length segments, and each segment is represented by a **frequency-domain feature vector** (essentially a compact encoding of that segment's spectral energy) <sup>62</sup>. For instance, a segment might be 1 second of EEG; BIOT computes features like band-power or an FFT of that second (the paper mentions "frequency energy vectors" per segment) <sup>63</sup>. These vectors are treated as tokens. The collection of tokens from all channels and times are then organized into what they call "sentences." In effect, BIOT creates a sequence of tokens that incorporates **spatio-temporal structure**: it uses learned **channel embeddings** (to indicate which channel a token came from) and **position embeddings** (to indicate the temporal order of tokens) <sup>63</sup>. By doing this, it can ingest data with varying lengths and channel counts – extra channels just mean more tokens tagged with a different channel embedding, longer recordings just produce longer token sequences. The linear Transformer encoder processes this token sequence and produces latent representations for each token. There is no separate decoder; the model is trained with a contrastive objective (described below), so the encoder's outputs (or a pooled representation of them) are the focus.

**Pretraining:** BIOT was pretrained using **self-supervised contrastive learning** rather than masked reconstruction <sup>60</sup>. Specifically, it adopted a *SimCLR-style* approach: the model generates embeddings for two augmented versions of the same EEG data and is trained to maximize their agreement (and minimize similarity to other, different signals) <sup>64</sup>. One of the augmentations mentioned is "masked inputs" – BIOT likely masks out parts of the input for one view and applies other augmentations (like adding noise or altering channels) for the other view, then pulls the two resulting embeddings together in the latent space <sup>65</sup>. The idea is that the encoder must produce a robust representation that is invariant to those augmentations, thus capturing the underlying brain-signal features. This contrastive objective did not require a dedicated decoder or reconstructing signals, which helped keep the model small and training efficient. BIOT's training data amounted to ~312k channel-hours of *scalp EEG*, aggregated from multiple sources (the authors aimed for cross-dataset generality) <sup>66</sup> <sup>67</sup>. It specifically was designed to handle **cross-dataset learning**: e.g., mismatched channels (one dataset might have 64 channels, another 19 channels; BIOT can ingest either by treating them as token sequences) and variable-length recordings <sup>63</sup>. In evaluations on clinical tasks (like seizure detection, seizure type classification, EEG pathology detection), BIOT performed impressively: even *without* any pretraining (randomly initialized BIOT fine-tuned on task) it already matched or beat many fully supervised baselines, and with pretraining it improved further <sup>68</sup>. This suggests the architecture's inductive bias (tokenizing into frequency features) is very effective for EEG even on its own <sup>68</sup>.



**Strengths:** BIOT's biggest advantage is **efficiency and flexibility**. It has only 3.3M parameters <sup>60</sup>, which is tiny compared to others – this makes it feasible to deploy on limited hardware or include as a sub-module without drastically increasing a multimodal model's size. Its linear attention means inference scales to long recordings and many channels in near-linear time. The tokenization approach addresses the issue of heterogeneity head-on: BIOT natively supports different channel configurations and recording lengths <sup>63</sup>. For a multimodal system that might see EEG from different devices or sessions, BIOT could adapt without retraining new positional encodings. The use of frequency-domain features per token is a strength in that it gives the model a condensed but informative input (especially in EEG, where a lot of important information is in the spectral domain). This likely helps with generalization across datasets that might have different noise or temporal characteristics but share spectral signatures for phenomena (like seizures have high-frequency components regardless of dataset). BIOT explicitly injects spatial information via channel embeddings (and even encodes electrode layout if known) to preserve topo-logical info <sup>69</sup>. As a result, it was one of the first models to truly perform **cross-dataset** EEG learning: the authors report that a single BIOT model fine-tuned on combined data with varying montages can outperform dataset-specific models <sup>70</sup> <sup>71</sup>. Another strength is that training via contrastive learning means BIOT's embeddings are explicitly organized in a semantic space (similar signals cluster together), which could be advantageous if one wants to align EEG embeddings with another modality's embeddings (a form of multimodal contrastive alignment could be done). BIOT's code is available (the authors' GitHub and integration into Braindecode library), and being a smaller model, it's easier to experiment with.

**Weaknesses:** BIOT's compact size means it may not capture the full complexity of EEG dynamics that larger models do. While it performed strongly on certain tasks, on others (e.g., one result noted it lagged on a MoBI dataset for a particular task) it wasn't always SOTA <sup>72</sup> <sup>73</sup>. There is likely a ceiling to how well a 3.3M model can do on very complex tasks or very subtle EEG patterns. Also, BIOT's reliance on frequency-segment tokens could discard some fine time-domain details (e.g., an abrupt spike that's spectrally broad might not be distinctly represented by coarse band power vectors). Contrastive training can sometimes yield less directly interpretable features compared to reconstruction-based training – we know BIOT's embeddings separate classes, but it's a bit less clear what each token's representation corresponds to physically. For use in a multimodal system, one challenge is that BIOT doesn't inherently output a single sequence organized by uniform timesteps; it outputs a set of tokens for all channels and times. To integrate with video frames (time steps), one might need to *reimpose* a structure on BIOT's output (e.g., group tokens by time). BIOT does maintain temporal order via position embeddings, but since it flattens all channels into one long sequence, tokens are ordered like: [Ch1 segment1, Ch1 segment2, ..., ChN segment1, ChN segment2, ...] or some pattern. If not careful, a naive use of BIOT's output could treat each token equally and not align them to specific global timesteps. In practice, one can reconstruct timing by knowing the segment length and order. But it's an extra step to get (B,T,D): you might have to collect all tokens that fall into a certain time window across channels and aggregate them (similar to CBraMod's approach). Another limitation is that BIOT was only trained on scalp EEG, and mostly clinical datasets – its performance on say, high-density EEG or very short events might not be as strong. Lastly, while contrastive pretraining is powerful, it might require careful fine-tuning (e.g., you often add a projection head for contrastive training that you might drop for fine-tuning). Ensuring you use the right embedding (before or after projection) for downstream integration matters.

**Output:** BIOT's encoder produces a sequence of token embeddings for each input sample. If an EEG recording yields (for example) 19 channels × 10 segments per channel = 190 tokens, then the output is of shape  $(B, 190, d)$  (with  $d$  presumably a few hundred; not explicitly stated, but given 3.3M total parameters,  $d$  might be on the order of 128 or 256 with a few layers). These tokens do not directly

correspond to unified timesteps across channels, so to get to  $(B,T,D)$ , we need to post-process. One approach: decide on a global time step (say 1-second intervals if segments were 1s). For each time step  $t$ , gather all token embeddings whose segment covers that time (each channel will contribute one token per time slot). Then you can aggregate those via mean or an attention pooling to get one vector for time  $t$ . Essentially, this is akin to *inverse tokenization*: BIOT tokenized by channel, so we de-tokenize by time. Another approach is to modify BIOT slightly: one could treat the output [CLS] token (if any) or an average of all tokens as a global summary – but that loses time resolution, not ideal for sequence modeling with video. Alternatively, one could fine-tune BIOT to output a sequence in temporal order (for example, feed tokens sorted by time rather than by channel into the Transformer). Because linear attention can handle longer sequences, you might even sort tokens by their timestamp (interleaving channels) so the output sequence is chronological. This wasn't needed for BIOT's original tasks, but as an adaptation strategy, it's feasible. For the simplest solution: use 1-second segmentation (which BIOT already does) and assume one embedding per channel per second. If you then average across channels, you get one embedding per second –  $(B,T, d)$  with  $T$  in seconds. This can then be upsampled or integrated with video frames (one could hold the EEG embedding constant for all frames within that second, or allow the cross-attention to attend multiple frames to the same EEG embedding). Given the gating cross-attention can handle mismatched lengths, you might feed the EEG sequence of length  $T_{\text{EEG}}$  and video of length  $T_{\text{video}}$  and let the model align implicitly (or use positional encoding to inform alignment). In summary, while BIOT doesn't naturally output a nicely indexed time sequence, a small adaptation (grouping channel tokens per time) can yield the needed  $(B,T,D)$  format.

**Code/Checkpoints:** The BIOT code is available on GitHub (under [ycq091044/BIOT](#))<sup>74</sup> and it's implemented in PyTorch. The authors even provide a pretrained checkpoint ("EEG-six-datasets-18-channels.ckpt" as per the repo) which likely corresponds to a model pretrained on a union of six EEG datasets with 18 channels common – possibly already aligned to a 18-channel montage for easier use<sup>74</sup>. There is also integration in the Braindecode library (an open-source EEG ML toolkit) which means it's relatively straightforward to use BIOT's encoder in Python code. The checkpoint can be loaded to get the pretrained weights for fine-tuning or embedding extraction. One consideration: if your EEG data have significantly different channels than those used to train (say you have 32 channels while the model was trained on 18), you may need to supply new channel embeddings or fine-tune with the new channels – but the architecture allows it, you just might have to initialize embeddings for any channel IDs that weren't seen before. Overall, BIOT's small size and accessible implementation make it a convenient baseline or component for EEG feature extraction in a larger system.

## NeuroGPT (Towards a Foundation Model for EEG)

**Architecture:** NeuroGPT (2023) is a mid-sized foundation model that uniquely marries a **bidirectional encoder** with a **GPT-2 style autoregressive decoder**<sup>75</sup>. The encoder ingests multi-channel *raw time-series EEG* (specifically 22-channel scalp EEG in the standard 10–20 montage) and is composed of a combination of **convolutional layers and Transformer layers**<sup>75</sup>. The conv layers likely serve to extract low-level features and downsample the sequence (common in time-series Transformers to reduce length), and then Transformer layers build contextual representations. Instead of decoding in the time domain, NeuroGPT's trick is to **mask some of the encoder's latent outputs** and then use a GPT-2 decoder to predict those missing latent embeddings in an autoregressive manner<sup>75</sup>. Essentially, it performs masked modeling in the latent space: the encoder produces a sequence of latent vectors summarizing the EEG, and some fraction of these are removed; the decoder (a causal Transformer like GPT-2) then generates them one by one as if it were "imagining" the missing neural signal evolution<sup>75</sup>. This approach is somewhat analogous

to masked language modeling with a sequenced decoder, ensuring the decoder sees previous unmasked latents and its own past outputs. Notably, the decoder in NeuroGPT is **larger than the encoder** (GPT-2 decoders are parameter-heavy) <sup>76</sup>, which is an uncommon design in EEG models (most others use lightweight decoders). The intuition is that the powerful decoder forces the encoder to create a good compressed representation that can be generatively modeled to reconstruct the signal. The overall param count is ~79.5M, with the encoder+decoder together <sup>75</sup>.

**Pretraining:** NeuroGPT was pretrained on the **entire Temple University Hospital (TUH) EEG Corpus**, which is ~541k channel-hours of clinical scalp EEG (around 27k hours of recordings with 19–22 channels) <sup>75</sup>. This massive dataset covers various conditions (it's a general hospital EEG archive), giving the model a broad exposure to normal and abnormal EEG patterns. The pretext task, as described, was masked latent reconstruction: randomly mask some of the encoder's output tokens and train the decoder to autoregressively predict them in order <sup>75</sup>. Because of the autoregressive decoding, the model has a generative flavor—NeuroGPT could in theory be used to sample or extrapolate EEG signals in latent space, not just encode them. After pretraining, the authors evaluated NeuroGPT on a downstream Motor Imagery classification task (a 4-class BCI dataset) <sup>77</sup>. This was a challenging test because the MI dataset had a *different electrode montage* (channels didn't match the 22-channel 10–20 scheme). To handle this, they applied an “*inverse-forward mapping*” – essentially re-referencing or mapping the MI EEG data onto the 10–20 layout used in training <sup>77</sup>. This indicates NeuroGPT's encoder expected a fixed set of channels (22 specific ones), unlike CBraMod or BIOT which allow variable channels. The results showed that using NeuroGPT's pretrained encoder (with either fine-tuning or frozen + linear head) significantly outperformed training an EEG classifier from scratch, and even beat some other supervised deep models designed for MI <sup>78</sup>. This verified that the representations NeuroGPT learned from TUH EEG were transferable to a very different task (BCI motor imagery), as long as the channel mismatch was addressed.

**Strengths:** NeuroGPT leverages an **extremely large EEG dataset** (the full TUH) for training, which is a big strength – it has essentially seen an “Internet-scale” amount of EEG (lots of patients, pathologies, etc.). This likely endowed it with a strong general ability to distinguish EEG patterns (like artifact vs brain signal, various waveform morphologies, etc.). The hybrid encoder-decoder architecture means it learned to compress EEG in a way that is useful for *generative reconstruction*, which could imply its latent space is more structured (the decoder has to make sense of it) compared to purely predictive encoders. In other words, the encoder's output might be a good candidate for a *shared embedding space* that could align with other modalities (since a GPT-2 decoder can treat it like a “language” of brain states). Another strength is that by using convolutional front-end, it might capture local patterns (spikes, waves) better than a pure Transformer, and by using autoregressive decoding, it inherently models temporal causality (which could help for tasks requiring sequence prediction). NeuroGPT's size (79M) is moderate – larger than BENDR or BrainBERT, but smaller than giants like Brant or Brant-2. This makes it somewhat easier to fine-tune than a 500M model, while still having enough capacity. The code for NeuroGPT is open-source <sup>75</sup> and it has gained some popularity (stars on GitHub indicate it's being tried out), meaning it's accessible. In summary, NeuroGPT stands out by combining **discriminative and generative training**, potentially giving it more versatile embeddings.

**Weaknesses:** One limitation is the **fixed channel assumption**. Unlike some newer models, NeuroGPT was not inherently designed to handle varying channel sets – the need to do an inverse-forward mapping for MI data means if your EEG channels don't match what it trained on, you must preprocess your data to fit the model. This could be problematic in multimodal settings if, say, you use a different montage; however, it's solvable by learning a mapping or zero-filling missing channels, etc. Another weakness is that the GPT-2

decoder, while powerful, adds a lot of parameters that are perhaps not needed when integrating into a multimodal model. If one only wants EEG embeddings, the decoder part is superfluous after pretraining (you'd likely drop it and just use the encoder). So in effect, a large chunk of the model isn't used at inference time for feature extraction – it was helpful in training but is baggage afterwards. This makes NeuroGPT somewhat inefficient if you can't utilize the decoder (though one could imagine using the decoder for EEG data augmentation or generation tasks). Additionally, NeuroGPT was only explicitly tested on one downstream task in the original work (and a fairly small one at that), unlike others that tried many tasks <sup>78</sup>. This means its universal effectiveness isn't as thoroughly validated; maybe it would also do well on sleep staging or emotion, but we have less proof. Its architecture, involving conv and transformer mixture, is a bit complex to modify – for example, if you wanted to change input sampling rate or length, you'd need to adjust conv layers accordingly. Another consideration: because the decoder looks one-step-ahead autoregressively, the encoder might have been optimized to produce *slightly redundant* representations (so that missing ones can be predicted from neighbors); whether this property is beneficial or not for direct feature use is unclear. It could be a strength or just a quirk. Finally, at 79M with a heavy decoder, training NeuroGPT from scratch is non-trivial, but using the pretrained weights mitigates that.

**Output:** NeuroGPT's encoder produces a sequence of latent embeddings for the EEG input. If the input is a fixed-length segment (say the model uses 20-second windows, divided into patches), the encoder might output, for example, 10 latent vectors (if using 2s patches) or some fixed sequence length representing that window. The decoder then takes those and generates missing ones, etc. For usage as an embedding model, one would typically take the encoder's output sequence **before** the decoder. This could be, for instance, shape  $(B, L, d)$  where  $L$  is the number of latent tokens per window. The hidden size  $d$  in NeuroGPT isn't explicitly stated, but GPT-2 small uses  $d=768$ , and likely the encoder matches that so that the decoder can attend to it. So assume  $d \approx 768$  and  $L$  might be on the order of 50 (if each second of EEG became one latent, 20s yields 20 tokens; or if they did something like downsample to 4Hz latent rate, maybe ~80 tokens per 20s). To integrate into (B,T,D) format, one approach: treat each latent as a "time-slice" embedding akin to patch embeddings. If NeuroGPT's latent corresponds to, say, 2 seconds of EEG (since they mentioned patch size 2s for others) <sup>79</sup>, then each latent = 2s summary. Those can align with a 2-second segment of video (if video is 30Hz, that'd be 60 frames corresponding to one latent). You can either repeat that latent for 60 frames or have the cross-attention attend the same latent for all frames in that interval. If more granularity is needed, one could fine-tune the encoder to output a latent per 1 second or 0.5 second by adjusting the conv stride or patching scheme. Another strategy is to *use the decoder to generate a higher frequency representation*: e.g., feed the encoder output and let the GPT-2 decoder generate intermediate latent states at smaller intervals – though this is complex and probably unnecessary for our purpose. Given Gated X-Attention likely can handle sequences of different lengths, you might simply input the sequence of latent embeddings from the encoder (length  $L$ ) and let it cross-attend to the sequence of video frames (length  $T_{\text{video}}$ ). Since  $L$  might be much smaller than  $T_{\text{video}}$ , the cross-attention will have to attend broadly (each EEG latent covers multiple frames). This can still work; the gating mechanism can learn to distribute video's attention across the appropriate EEG latents covering that time. If needed, one can linearly interpolate the EEG embeddings to have the same  $T$  as video frames (but that might smear things). In summary, NeuroGPT can output an embedding sequence at a lower temporal resolution than video; aligning them may involve upsampling EEG or downsampling video timeline, depending on design choices.

**Code/Checkpoints:** NeuroGPT's code is on GitHub (GPL-3.0) <sup>80</sup>. It has instructions to load the TUH pretrained model. The repository includes data preprocessing (especially for TUH) and training pipelines, so it's quite useful if one wants to replicate or fine-tune it. For applying to new data, you'd need to preprocess

your EEG to 22-channel 250Hz (the TUH standard) and segment it appropriately. Checkpoints for the pretrained model are provided, so you can directly load the weights. If you plan to use only the encoder, you might have to extract the encoder part from the state dict and integrate it into your multimodal model's forward pass. Since the license is GPL, there might be some considerations if merging into a larger codebase (GPL requires derivative code to be GPL as well), but for internal research it's not an issue. Documentation is decent, and there's community interest which means issues/answers might be available for common questions.

## Comparison of EEG Foundation Models

The table below summarizes key characteristics of each model:

| Model                                                  | Architecture & Size                                                                                                                                                   | Pretraining Data & Objective                                                                                                                                                                                  | Output                                                                                                                                                                             | Strengths                                                                                                                                                                                                                                                          | Weaknesses                                                                                                                                                                                                                         | Code/W                                                            |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>BrainBERT</b><br>(Wang et al. 2023)<br><sup>3</sup> | Transformer encoder (BERT-like) + 2-layer linear decoder; ~43 M params <sup>1</sup> . Operates on single-channel spectrogram patches (per EEG channel) <sup>2</sup> . | ~4.5k channel-hours iEEG (intracranial) from Brain Treebank <sup>3</sup> . Self-supervised <b>masked spectrogram patch reconstruction</b> (masked 10% of time-freq patches) <sup>4</sup> .<br><sup>81</sup> . | Sequence of embeddings per channel-spectrogram (e.g. per 5s window's patches). Hidden dim on order of 256–512 (not specified). Yields one embedding vector per masked patch/token. | - First EEG foundation model; improved sample efficiency (linear probe ~ full model performance) <sup>10</sup> .<br>- Generalizes to new subjects/electrode locations in same domain <sup>7</sup> .<br>- Incorporates frequency information via spectrogram input. | - Trained solely on iEEG (limited domain) <sup>9</sup> ; may not transfer directly to scalp EEG.<br>- Ignores cross-channel spatial info (each channel independent).<br>- Moderate model size; smaller capacity than newer models. | Yes – Av<br>on GitHu<br>(MIT).<br>Pretrain<br>weights<br>provided |

| Model                                      | Architecture & Size                                                                                                                                                                                                                                                                  | Pretraining Data & Objective                                                                                                                                                                                    | Output                                                                                                                                                        | Strengths                                                                                                                                                                                                                                                                                                                                | Weaknesses                                                                                                                                                                                                                                 | Code/W                                                                     |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>CBraMod</b><br>(Wang et al. 2025)<br>12 | <b>Criss-cross Transformer</b><br>encoder (12 layers, 8 heads split into 4 spatial + 4 temporal) 13<br>14 . Patch-wise conv embedding (1 s patches per channel) + ACPE positional conv. ~ tens of M params (hidden dim 200) 17 . No decoder except a linear head for reconstruction. | >9,000 h of scalp EEG (TUH corpus, 15k subjects) 19<br>82 . Self-supervised <b>patch-level masked EEG reconstruction</b> (like MAE) 16 . Masks ~40% of patches; reconstruct only masked ones (MSE loss) 18 21 . | Embedding for each patch = each channel's 1 s segment. Output: (Batch, Channels×T_patches, 200). Can be pooled to (B, T_segments, 200) for time-based output. | - Separates spatial vs. temporal attention – respects EEG's structure 12 .<br>- <b>Format-flexible:</b> handles variable channel layouts via ACPE 83 .<br>- SOTA performance on 10 diverse EEG tasks (BCI, clinical) – very generalizable 16 .<br>- Large training corpus, non-invasive data; robust to noise after heavy preprocessing. | - Output is many patch embeddings (one per channel per time); needs aggregation for per-time representation.<br>- High token count for long signals (may be memory-intensive).<br>- New model (2025); less proven outside published tasks. | Yes – Open source Python code 22<br>Pretrained model available (ICLR 2025) |

| Model                                                   | Architecture & Size                                                                                                                                                                                                                                                                   | Pretraining Data & Objective                                                                                                                                                                                                                                  | Output                                                                                                                                                                                         | Strengths                                                                                                                                                                                                                                                                                                         | Weaknesses                                                                                                                                                                                                               | Code/W                                                                                                  |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Brant</b><br>(Zhang et al. 2023)<br><small>9</small> | Dual Transformer encoders (temporal + spatial) + linear decoder <small>26</small> .<br>Largest version ~500 M params; also 68M/104M/249M variants <small>30</small> .<br>Processes raw multi-channel time series; incorporates time and <b>frequency</b> features <small>28</small> . | 281k channel-hours iEEG (SEEG from 9 patients) <small>9</small> .<br>Self-supervised <b>masked time-series reconstruction</b><br>– predict missing multichannel signals <small>29</small> .<br>Pretrained on one intracranial dataset (long-term recordings). | Learned hidden representation per channel and time. Output: contextual features for each time step and channel (before decoding). Can yield one embedding per time step when spatially pooled. | - Captures long-term temporal patterns + cross-channel correlations via dedicated spatial encoder <small>26</small> .<br>- Demonstrated SOTA on forecasting, imputation, seizure detection (iEEG tasks) <small>33</small> .<br>- Variable-channel input (adapts to different electrode setups) <small>9</small> . | - Narrow training domain (single iEEG dataset) – limited diversity <small>9</small> .<br>- Very large model (500M) – resource intensive.<br>- Primarily validated on intracranial data; transfer to scalp EEG not shown. | Yes – Co<br>GitHub;<br>weights<br>(includin<br>500M) re<br><small>36</small> . Go<br>docume<br>(project |

| Model                                                        | Architecture & Size                                                                                                                                                                                                                                                                                   | Pretraining Data & Objective                                                                                                                                                                                                                                                                                                                                                         | Output                                                                                                                                                                                                            | Strengths                                                                                                                                                                                                                                                                                                                                                                                                              | Weaknesses                                                                                                                                                                                                                                                                           | Code/W                                                                                                                    |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Brant-2</b><br>(Yuan et al. 2024)<br><small>84 85</small> | <i>Enhanced Brant:</i> Transformer encoders (temporal then spatial) with <b>multi-branch input (time+freq)</b> and augmentation module <small>59 46</small> . ~1 <b>billion</b> params <small>86 49</small> (largest EEG model so far). Encoder uses RMSNorm, Swish; supports multi-task pretraining. | ~4 TB of mixed EEG + SEEG data (>15k subjects, many datasets) <small>48 49</small> . <b>Dual objectives:</b> masked patch prediction <i>and</i> future patch forecasting <small>50 40</small> . (Reconstruct missing parts and predict next sequence quarter.) Extensive data augmentation (vary sampling rate, channel count) during training for robustness <small>41 42</small> . | Sequence of patch embeddings (context length ~16 patches). E.g. outputs 16 latent embeddings for ~30 s window (if patch=2 s). Hidden size on order of 768–1024. One vector per patch (can cover seconds of data). | - <b>Extremely diverse pretraining</b> – both scalp and intracranial; captures wide range of neural patterns <small>48 49</small> .<br>- Largest capacity → SOTA on many tasks; handles low-label regimes well <small>56</small> .<br>- Built-in resilience to sampling and montage differences (via augmentation) <small>41</small> .<br>- Learns temporal dynamics (via forecasting) + content (via reconstruction). | - Huge model (1B) – very high computational cost (training and inference) <small>53</small> .<br>- Fixed patch size/context; may need adjustment to align with fine-grained video timeline.<br>- Possibly overkill if only scalp EEG needed; may require simplifying for deployment. | Yes – Co<br>pretrain<br>weights<br>released<br>(Apache<br>Well-<br>docume<br>Might ne<br>model-p<br>setup fo<br>largest v |



| Model                                           | Architecture & Size                                                                                                                                                                                                                                                                                                                          | Pretraining Data & Objective                                                                                                                                                                                                                                                                                                       | Output                                                                                                                                                                                                                 | Strengths                                                                                                                                                                                                                                                                                                                                                                                      | Weaknesses                                                                                                                                                                                                                                                                                                                                                         | Code/W                                                                                                 |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>BIOT</b><br>(Yang et al. 2023) <sup>60</sup> | Small <b>linear-transformer</b> encoder (efficient attention); ~3.3 M params <sup>60</sup> . Custom <b>tokenizer</b> : each channel's signal split into segments → per-segment frequency-energy vector tokens <sup>63</sup> . Positional and channel embeddings added <sup>63</sup> . No decoder; uses projection head for contrastive loss. | 312k channel-hours of scalp EEG (multiple clinical datasets) <sup>60</sup> . Self-supervised <b>contrastive learning</b> (SimCLR-like): maximize agreement between two augmented views of the EEG (e.g. masked vs. unmasked) <sup>65</sup> . No direct signal reconstruction – learns via instance discrimination across datasets. | Sequence of token embeddings, length = sum of segments over all channels. E.g. 19-ch EEG, 1 s segments → 19 tokens per second. Hidden size relatively small (e.g. 128–256). Can pool tokens per time or per recording. | - <b>Very efficient &amp; lightweight</b><br>– suitable for low-resource use. <br>- Handles variable length and channel count natively <sup>63</sup> (flexible “biosignal sentence” input). <br>- Incorporates spectral features explicitly → robust across datasets. <br>- Strong performance on cross-dataset evaluation (e.g. seizures, pathology) even without pretraining <sup>68</sup> . | - Lower capacity limits performance on very complex tasks (lagged behind larger models on some benchmarks) <sup>73</sup> . <br>- Outputs are per-channel segments, not naturally aligned to global time steps (requires post-processing to integrate channels). <br>- Contrastive objective may require careful fine-tuning (no direct reconstruction of signals). | Yes – Co<br>GitHub<br>(PyTorch<br>Braided<br>integrati<br>Pretrain<br>weights<br>available<br>(NeurIPS |

| Model                                                  | Architecture & Size                                                                                                                                                                                                                                                                                                          | Pretraining Data & Objective                                                                                                                                                                                                                                                                                                                                            | Output                                                                                                                                                                                      | Strengths                                                                                                                                                                                                                                                                                                                                                                                              | Weaknesses                                                                                                                                                                                                                                                                                                                     | Code/W                                                                                                          |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>NeuroGPT</b><br>(Zhao et al. 2023)<br><sup>75</sup> | Encoder–Decoder: Conv + Transformer <b>encoder</b> (to encode 22-ch raw EEG) + <b>GPT-2 decoder</b> (autoregressive) <sup>75</sup> . ~79 M params total. Encoder produces latent sequence; decoder predicts masked latent positions in sequence (causal order) <sup>75</sup> . Essentially a masked latent generative model. | 541k channel-hours of scalp EEG (TUH corpus, ~27k hours) <sup>75</sup> . <b>Masked latent modeling:</b> encode full sequence, mask some latent tokens, use GPT-2 to generate them sequentially <sup>87</sup> . Learns to represent EEG such that a language-model can “fill in” missing pieces. Evaluated on motor imagery BCI (after mapping channels) <sup>77</sup> . | Encoder outputs a sequence of latent embeddings (length depends on window, e.g. one per 2 s segment). Hidden dim ~768 (matching GPT-2 small). One vector per latent token (per time patch). | - Large-scale scalp EEG training → good general EEG feature extractor <sup>78</sup> .<br><br>- Combines discriminative and generative learning; embeddings are structured for both understanding and prediction.<br><br>- Convolution + Transformer encoder can capture local and global patterns. <br>- Proven transfer to a very different task (EEG motor imagery) with fine-tuning <sup>78</sup> . | - Assumes a fixed 22-channel 10–20 input (needed remapping for new montage) <sup>77</sup> . <br>- Decoder heavy; not used at inference if only embeddings are needed (makes model bigger than necessary for embedding tasks). <br>- Fewer downstream evaluations in literature (used mainly MI classification to demonstrate). | Yes – Open source (GPL-3.0); GitHub; pretrained weights TUH model available; Documentation and scripts provided |

**Table Legend:** *Channel-hours* = total hours × number of channels (sum over recordings). **SEEG** = stereotactic EEG (intracranial). **MAE** = Masked Autoencoder. **SOTA** = state-of-the-art.

## Recommendations for EEG Embeddings in a Multimodal (EEG + Video/Audio) Model

**Best Candidates:** For initializing an EEG embedding module to pair with video/audio, **CBraMod** and **Brant-2** emerge as top choices, with **BIOT** as a lightweight alternative in constrained settings. These

models offer a combination of strong generalization, well-documented implementations, and architectural features that facilitate multimodal integration:

- **CBraMod** <sup>12</sup> <sup>16</sup> : CBraMod's criss-cross architecture is well-suited for producing time-aligned EEG embeddings. Because it explicitly separates temporal processing (within each channel) and spatial processing (across channels at a given time), it can naturally yield an embedding **per time segment** that encapsulates all channels. This is ideal for multimodal alignment – e.g. a 1-second CBraMod segment embedding can be aligned with 1 second of video/audio. Architecturally, CBraMod's design mirrors how one might manually fuse EEG channels for a given time slice, which means after pretraining, one can insert a simple pooling operation to get  $(B,T,D)$  outputs: CBraMod has essentially *already done the heavy lifting of mixing channels* at each time step through its spatial attention <sup>13</sup> <sup>88</sup> . Another advantage is its **positional encoding flexibility (ACPE)** <sup>83</sup> – if your EEG setup changes (say different number of electrodes in a new dataset), CBraMod can adapt with minimal fuss, an important factor if the multimodal model might be applied in varied recording conditions. It's implemented in PyTorch with clean code and comes from an academic group (ICLR 2025 paper) with thorough experiments, so using it as a starting point is straightforward <sup>22</sup> . In multimodal training, one could initialize the EEG branch with CBraMod's pretrained weights (maybe excluding the final linear reconstruction head), and then add a pooling layer to produce one vector per time step. For example, you could use **channel-wise mean pooling per time** or even a learned gated pooling to condense the 19 channel-wise patch embeddings into one – thanks to CBraMod's prior spatial attention, this pooling will still yield a rich representation of the whole brain state at that time. Those  $(B,T,D)$  EEG embeddings can then feed into a Gated Cross-Attention block alongside video frames. Gated attention (as used in e.g. Flamingo models) can handle sequences of different lengths by attending over the available EEG tokens for each video token. Since CBraMod was trained with 1-second patches, if your video frames are, say, 30 Hz, you might either use 1 EEG token per second and let each 30 video frames attend the same EEG token, or you could upsample the EEG tokens linearly to 30 per second (though the former is simpler and likely sufficient). In summary, **CBraMod offers an excellent balance of performance and usability**: it's powerful (SOTA on many EEG tasks) <sup>22</sup> , not overly large (~26M params, manageable to fine-tune), and its architectural inductive bias (separate spatial/temporal) directly aligns with creating per-time-step embeddings for multimodal fusion.
- **Brant-2** <sup>84</sup> <sup>85</sup> : If maximizing performance is the priority and computational resources are available, Brant-2's pretrained model is arguably the most comprehensive EEG foundation model to date. Its embeddings are trained on an extremely diverse set of conditions – this could be crucial if the multimodal model needs to handle a wide range of brain states (from normal to pathological, awake to sleep, etc.). **Architectural advantages for Brant-2 in multimodal context**: (1) It integrates **time and frequency** features for each patch <sup>44</sup> , which means the EEG embedding it produces carries multi-scale information (this could help the cross-modal model because some audiovisual patterns align more with EEG frequency content – e.g., a rhythmic visual stimulus might correlate with a specific EEG frequency band; Brant-2's representation can capture that explicitly). (2) It has a **temporal encoder and a spatial encoder** – conceptually similar to CBraMod's approach, though sequential rather than parallel. This two-stage encoding means by the time you get to the output, each patch's embedding has seen both its local temporal context and cross-channel context <sup>59</sup> <sup>46</sup> . You can thus take Brant-2's output for each time patch as a holistic representation of the EEG at that period. (3) The **forecasting objective** in Brant-2's pretraining gives it a form of temporal coherence that could align well with how video frames progress over time. It might, for instance,

produce embeddings that smoothly transition in time (since it had to predict the next ones), which could be beneficial for temporal alignment with continuous video. Practically, using Brant-2 in a multimodal model would involve loading its encoder (temporal+spatial) and perhaps freezing it initially (given its size) or fine-tuning lightly due to domain shift (if your data is a bit different). One would then define the EEG output per patch as a sequence for cross-attention. Likely, we'd set the patch length in Brant-2 equal to a fixed interval that also makes sense for video (perhaps 1 second or 2 seconds per patch). We can then align each patch with a corresponding set of video frames. For example, if patch = 2s, then every 2 seconds of video (say 60 frames) corresponds to one EEG token. A **gated cross-attention** can be set such that the video modality attends to the EEG tokens encompassing its timeframe. If more temporal granularity is needed, one could fine-tune Brant-2 with a shorter patch size (the architecture supports different patch lengths and even resampling augmentation, so it's flexible). Brant-2 is well-documented, and the availability of 1B-parameter weights trained on 4TB data is a huge asset – essentially giving your multimodal model a *ready-made encoding of the brain* learned from a vast swath of human data. The downside is inference cost; one might consider using the **smaller “Brant” (500M) or even Brant-2 distilled** if available, or freeze Brant-2 to avoid backprop overhead. Still, if results are paramount and deployment environment can handle it, Brant-2 initialized EEG embeddings will likely be the most expressive and robust.

- **BIOT (as a lightweight option):** If the multimodal model must be kept lean (e.g., for real-time or edge deployment), or as a quick initial baseline, BIOT's embeddings could be used. BIOT will give you **19 frequency-token embeddings per second** for a 19-channel EEG (assuming 1s segments). You could compress those 19 into one by averaging – effectively yielding a 1Hz embedding sequence. While this is a coarse representation, it does capture a summary of the EEG power in each band across the head for each second, which might correlate with low-frequency aspects of video (like scene changes, rhythm of actions, etc.). BIOT's strength is that it's easy to plug in and run (few parameters, fast). Its weakness is that it won't capture subtle time patterns within that 1s segment (since it reduces each second per channel to one vector). However, one could improve temporal resolution by using shorter segments (the code could be adjusted to, say, 0.5s segments, doubling token rate). BIOT is quite flexible with channel counts, so if, for example, you have a 32-channel EEG, you can still use BIOT by training new channel embeddings or using it as is (some channels might be ignored or you pad tokens). It also has the advantage of being **trained on multi-condition data** (multiple datasets), so it has some general knowledge. For a multimodal foundation model that will be heavily fine-tuned or trained further on a specific dataset, BIOT could serve as a good initialization to avoid starting from scratch, without imposing much overhead.

#### **Additional Considerations:**

- **Modality Alignment:** None of these EEG models has seen video or audio data, so their embeddings live in a brain-signal feature space that is not aligned with the other modalities. In the multimodal model, the Gated Cross-Attention will learn to relate EEG embeddings to video/audio embeddings. It may help to ensure all modalities' embeddings have the same dimensionality (which you can achieve by adding a projection layer to the EEG model if needed – e.g., project CBraMod's 200-d output to the 768-d used by a vision model). Also, consider the **temporal resolution mismatch:** video is high-frequency (e.g. 30 Hz frames), EEG meaningful patterns often at lower freq (say EEG model outputs at 1–10 Hz). A strategy is to allow the cross-attention to attend across time within a certain window. For instance, a video frame could attend not just the EEG token at the exact same time, but also a few neighboring EEG tokens (before/after) since EEG might lag or lead slightly

relevant to stimuli (neural response delays). Gated attention can modulate how much EEG influences the visual stream, so it can learn an optimal alignment or integration.

- **Potential Adaptation Strategies:** Beyond simple averaging or pooling of EEG channel embeddings per time, one could explicitly use a **small alignment model** on top of the EEG foundation model. For example, after CBraMod, add a **Gated Attention pooling** that takes all channel patches at time  $t$  and produces one vector (this is like an attention-based channel fusion, which could weight important channels more). This learned pooling can be trained during multimodal training. Another idea is to insert a **“fusion token”** per time step that attends to all channel patches (similar to a CLS token per time); CBraMod didn't have one by default, but you can append a dummy token to each time stripe and let it gather info – effectively doing what averaging would do, but in a trainable manner. This could yield a stronger per-time representation than naive averaging. In Brant-2's case, since it already does temporal then spatial sequentially, one could tap the output after the spatial encoder at each time patch – that is inherently a fused representation. If wanting a finer alignment (e.g., one EEG vector per video frame), you could **upsample via interpolation** or a small temporal decoder in the EEG branch that takes the coarse sequence and outputs intermediate vectors (maybe using the forecasting head of Brant-2 in a creative way, or a simple linear interpolation since adjacent EEG tokens will be similar).
- **Domain Specificity vs. Generality:** Since your multimodal model likely will be trained (or at least fine-tuned) on a specific dataset where EEG and video/audio are recorded together, there's a question: is it better to start with a highly general model (like Brant-2 or CBraMod on broad data) or something more specialized? Generally, a broad model gives robust low-level features and you fine-tune to the domain. But if, say, your application is emotion recognition, a model like CBraMod which saw emotion EEG in its pretraining (DEAP dataset, etc.) <sup>16</sup> might have a slight head-start in relevant feature extraction. Brant-2 saw lots of clinical EEG; if your task is more cognitive (like correlating EEG with video content), CBraMod's BCI-oriented training might align better with that kind of signals. On the other hand, Brant-2's sheer size might capture some of everything, including cognitive EEG patterns.
- **Video/Audio Model Considerations:** On the video side, if you use a transformer-based model for video (say a TimeSformer or ViT for frames) that outputs a sequence of frame embeddings, those will likely be at a much higher frequency than EEG tokens. Gated cross-attention can be implemented such that each video token attends to all EEG tokens (and possibly vice versa). A gating mechanism (like in Flamingo) will weigh how much to incorporate EEG information at each layer. To ensure effective learning, you might need to **normalize or scale** the EEG embeddings appropriately (so they're in a similar range as the video embeddings). Some practitioners concatenate modalities with learned scale parameters – but with cross-attention, as long as the queries and keys are scaled properly (maybe use LayerNorm on EEG branch outputs), it should learn the alignment.
- **Training strategy:** A practical strategy is to **freeze the pretrained EEG model for initial epochs** while training the video/audio parts, to avoid destabilizing the well-formed EEG features (especially if you have limited EEG-video paired data). Then gradually unfreeze and fine-tune the EEG model with a low learning rate. This ensures the rich representation from CBraMod/Brant-2 isn't lost due to random initialization of the video branch. Over time, the EEG model can adjust to better complement the video features (it might, for example, shift focus to certain frequency bands if the video stimuli are known to induce those). Both recommended models have shown that **fine-tuning is effective**

with relatively small data <sup>89</sup> <sup>57</sup>, so even if your multimodal dataset isn't huge, you should be able to adapt them without overfitting.

In conclusion, **CBraMod** is likely the best all-around choice for a starting point: it's modern, efficient, and specifically built to handle EEG idiosyncrasies (making integration smoother). **Brant-2** is the choice when maximum pretraining and representation power is needed – for instance, if your multimodal model is aimed at high-stakes or very general EEG-video tasks (like an AI that interprets any neural and visual data thrown at it), Brant-2's extensive pretraining will provide a strong foundation (at the cost of heavy computation). For rapid prototyping or low-compute scenarios, **BIOT** offers a decent initialization that you can later swap out for a heavier model if needed.

Each of these models comes with **solid PyTorch implementations and documentation**, which was a priority – CBraMod's GitHub, Brant-2's GitHub, and BIOT's repository are all accessible and include examples. This means you can relatively quickly integrate them into your codebase, verify that you're getting sensible EEG embeddings, and then focus on training the cross-modal attention to fuse EEG with video/audio. By leveraging these foundation models, you capitalize on a wealth of prior learning, likely accelerating convergence and boosting final performance of your EEG+Video/Audio foundation model.

---

1 2 3 4 5 6 7 8 9 10 26 29 30 31 32 35 60 61 62 63 64 65 66 67 68 69 70 71 72 73

75 76 77 78 79 81 87 89 EEG Foundation Models: A Critical Review of Current Progress and Future

Directions

<https://arxiv.org/html/2507.11783v2>

<sup>11</sup> [klab.tch.harvard.edu](https://klab.tch.harvard.edu)

<https://klab.tch.harvard.edu/publications/PDFs/gk8131.pdf>

12 13 14 15 16 17 18 19 20 21 22 23 24 25 82 83 88 [2412.07236] CBraMod: A Criss-Cross

BrainFoundation Model for EEG Decoding

<https://ar5iv.labs.arxiv.org/html/2412.07236v5>

27 28 33 34 36 Brant: Foundation Model for Intracranial Neural Signal | Daoze Zhang, Zhizhang Yuan, Yang Yang, Junru Chen, Jingjing Wang, Yafeng Li

<https://zju-brainnet.github.io/Brant.github.io/>

37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 84 85 86 Brant-2:

Foundation Model for Brain Signals

<https://arxiv.org/html/2402.10251v3>

<sup>74</sup> [ycq091044/BIOT: NeurIPS2023 - A generic biosignal learning ...](https://github.com/ycq091044/BIOT)

<https://github.com/ycq091044/BIOT>

<sup>80</sup> GitHub - wenhui0206/NeuroGPT: Code for the paper "Neuro-GPT: Towards a Foundation Model for EEG"

<https://github.com/wenhui0206/NeuroGPT>